FICTIONAL RESUME - FOR INTERVIEW PRACTICE AND SOFTWARE TESTING ONLY
All people, organizations, experience, projects, and results below are invented.

JORDAN EXAMPLE

AI / Machine Learning Engineering Intern

jordan.example@example.com | Portfolio: https://example.com/jordan-ml

SUMMARY

Computer science student interested in building reliable machine learning systems. Experience with Python, PyTorch, natural language processing, retrieval-augmented generation, and deploying model inference APIs. Comfortable comparing baselines, designing evaluations, investigating errors, and explaining engineering tradeoffs.

EDUCATION

Example Institute of Technology - B.S. in Computer Science

Expected graduation: May 2027 | GPA: 3.7/4.0

Relevant coursework: Data Structures, Algorithms, Linear Algebra, Probability,
Statistics, Machine Learning, Databases, and Software Engineering.

TECHNICAL SKILLS

Languages: Python, SQL, JavaScript, C++

Machine learning: PyTorch, scikit-learn, pandas, NumPy, Hugging Face Transformers

Topics: Classification, embeddings, model evaluation, RAG, data leakage, calibration

Backend and deployment: FastAPI, Docker, PostgreSQL, Git, GitHub Actions, Linux

Evaluation: Precision, recall, macro F1, confusion matrices, retrieval recall, latency

PRACTICE TARGET

AI/ML engineering internship focused on NLP, evaluation, and model deployment. Suggested interview topics: model choice, leakage prevention, error analysis, RAG grounding, precision/recall tradeoffs, CPU inference, and team communication.

EXPERIENCE

Machine Learning Engineering Intern - Northstar Demo Labs (fictional organization)

June 2026 - August 2026

- Built a classifier for 12,000 synthetic support tickets across eight categories.
- Compared a TF-IDF/logistic regression baseline with a fine-tuned DistilBERT model; improved macro F1 from 0.72 to 0.86 on a held-out synthetic test set.
- Grouped tickets by generation template before splitting the data to reduce leakage from near-duplicate examples; inspected confusion matrices by category.
- Served predictions through FastAPI with input validation and confidence-based abstention, routing uncertain predictions to a simulated review queue.
- Reduced p95 inference latency from 180 ms to 110 ms in a controlled CPU load test by batching tokenization and loading the model once at application startup.
- Documented failure cases, synthetic-data limitations, and reproducible experiments for a fictional team of three engineers.

PROJECTS

CampusGuide - Retrieval-Augmented Question Answering Demo

Python, sentence embeddings, FAISS, FastAPI

- Created a question-answering assistant over 60 invented campus policy documents.
- Implemented document chunking, embedding retrieval, source citations, and a response that abstains when the retrieved material does not support an answer.
- Improved retrieval recall@3 from 0.78 to 0.91 on 200 held-out synthetic queries by tuning chunk size and adding a reranking step using a separate development set.
- Tested conflicting documents, missing answers, and instruction-like text inside documents; treated retrieved content as data rather than system instructions.

SensorWatch - Time-Series Anomaly Detection

Python, PyTorch, NumPy, Docker

- Generated synthetic sensor streams containing drift, missing values, and spikes.
- Compared a rolling-statistics baseline with a small autoencoder, reporting precision and recall rather than accuracy because anomalies were rare.
- Selected the alert threshold on validation data, then achieved 0.84 precision and 0.89 recall on a held-out simulated test stream.
- Packaged the inference service in Docker and added tests for missing inputs, normalization consistency, and model-loading failures.

COLLABORATION

Student ML Study Group - Project Coordinator (fictional group)

September 2025 - Present

- Organized a four-person project team, reviewed pull requests, and maintained experiment notes so teammates could reproduce results.
- Presented a short workshop on overfitting, train/validation/test splits, and why strong synthetic benchmarks do not establish real-world performance.